MLOps Internal POE

UAT Tool Setup, Dry Run and Summary Report

Student: Shweta Patil **Roll Number:** _____ **Division:** A

Project: Hello MLOps UAT — Flask, GitHub, Jenkins, Docker, Pytest and Evidently

1. Objective

Configure a simple application in a UAT-style environment and demonstrate source control, automated testing, containerization, CI/CD orchestration, and data-drift reporting. The monitoring data is synthetic and intentionally shifted for demonstration.

2. Project Files

hello.py — Flask application; test_hello.py — Pytest tests; requirements.txt — Python dependencies; Dockerfile — application image; Jenkinsfile — Windows Jenkins pipeline; monitor.py — Evidently data-drift report; README.md — commands and setup instructions.

3. Tool Commands and Verification

Tool	Commands / Verification	Status
Python / Flask	python --version python hello.py Open localhost:5000 and /health	Run on your PC and record PASS/FAIL
Git / GitHub	git --version git status git push -u origin main	Repository checkout must be verified
Pytest	python -m pytest -v Expected: 2 tests pass	Run and capture actual output
Docker	docker --version docker build -t mlops-uat-hello:latest . docker run -d --name mlops-uat-hello -p 5001:5000 mlops-uat-hello:latest	Run and verify /health
Jenkins	Run the SmartShelfX-CICD or new POE pipeline and inspect Console Output	Record final build status
Evidently	python monitor.py Expected file: evidently_report.html	Run and open generated report

4. Pytest Expected Result

```
test_hello.py::test_home_returns_200_and_greeting PASSED
test_hello.py::test_health_endpoint_returns_healthy PASSED
2 passed
```

This is the expected result, not a captured execution log. Replace or supplement it with a screenshot of the actual test output after running the commands.

5. Evidently Monitoring

The monitor.py script compares two small synthetic datasets and generates evidently_report.html. The current dataset is intentionally shifted. This demonstrates the reporting workflow; it is not evidence of real production drift.

6. Evidence to Add Before Submission

Capture actual screenshots of: (1) VS Code project files, (2) Pytest terminal output, (3) running Flask page and /health response, (4) Docker image/container and health response, (5) Jenkins Console Output, and (6) Evidently HTML report. Fill in roll number and confirm all PASS/FAIL statuses based on actual runs.

7. Result

The program files and reproducible commands are prepared. The final PASS/FAIL result must be based on execution in the student's Windows/Linux UAT environment. Do not submit unverified status as a successful run.